

目录



- [01、Chrome DevTools MCP ...](#)
- [02、CDP 原理与消息流转](#)
- [03、默认接入与启动体验](#)
- [04、从 mcp.json 到工具注册](#)
  - [第一步，读取配置。](#)
  - [第二步，启动 server 进程。](#)
  - [第三步，MCP 握手。](#)
  - [第四步，拉取工具列表。](#)
  - [第五步，注册到 ToolRegistry。](#)
- [05、系统提示词升级](#)
- [06、场景实测](#)
- [07、PaiCLI 如何写到简历上](#)

目录

- [01、Chrome DevTools MCP ...](#)
- [02、CDP 原理与消息流转](#)
- [03、默认接入与启动体验](#)
- [04、从 mcp.json 到工具注册](#)
  - [第一步，读取配置。](#)
  - [第二步，启动 server 进程。](#)
  - [第三步，MCP 握手。](#)
  - [第四步，拉取工具列表。](#)
  - [第五步，注册到 ToolRegistry。](#)
- [05、系统提示词升级](#)
- [06、场景实测](#)
- [07、PaiCLI 如何写到简历上](#)

Search

原创

# Agent 终于能开浏览器了！Chrome DevTools MCP 接入全解析

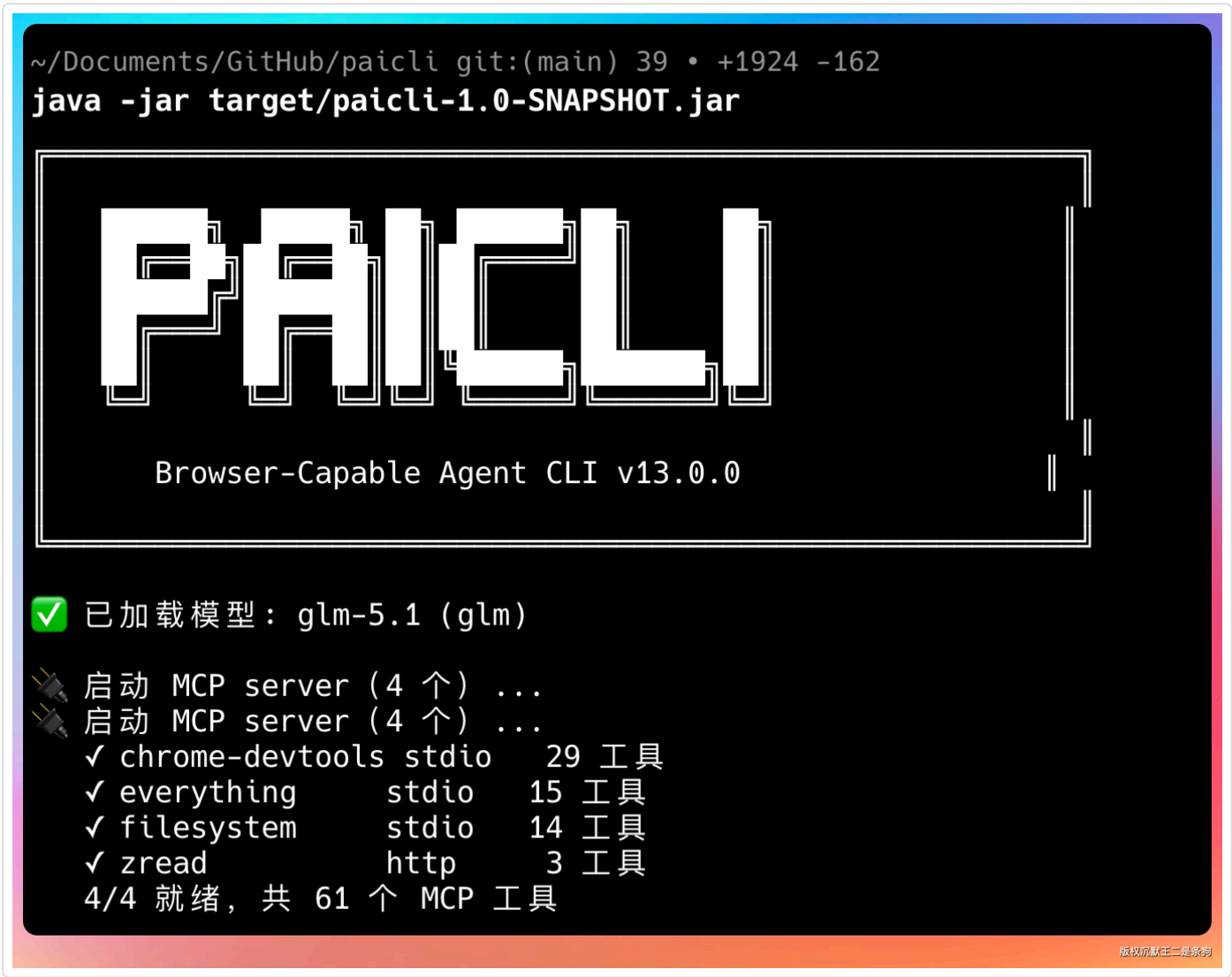
大家好，我是二哥呀。

做了联网搜索，做了 MCP，我发现 PaiCLI 还有一个问题，Agent “看不见”一些固有生态的内容，比如说微信的内容。

直接用 web\_fetch 去读微信的内容，是读不到的，因为微信生态的内容，外部的搜索引擎无能为力。

还有一些动态渲染的网页内容，抓回来的 HTML 里几乎没有内容。

以及想让 Agent 填表单、截图、看看控制台报错等，更是想都别想。



这一期，我们来接入 Google 官方的 Chrome DevTools MCP，让 PaiCLI 从“能调工具的 Agent”进化成“能开浏览器的 Agent”。

能导航页面、点击元素、填表单、拿 DOM 快照、看网络请求。

看完这一篇，大家就会明白一套完整的浏览器接入方案是怎么设计的，从启动体验到人工审批，从提示词策略到多模态边界，每一步都有讲究。

有了 Chrome Devtools MCP 后，Agent 就可以主动 fallback 到浏览器 MCP：调 `mcp__chrome-devtools__new_page` 打开页面，等文章容器加载完，再调 `take_snapshot` 拿 DOM 文本快照，最后总结输出。

## 目录

- [01、Chrome DevTools MCP ...](#)
- [02、CDP 原理与消息流转](#)
- [03、默认接入与启动体验](#)
- [04、从 mcp.json 到工具注册](#)
  - [第一步，读取配置。](#)
  - [第二步，启动 server 进程。](#)
  - [第三步，MCP 握手。](#)
  - [第四步，拉取工具列表。](#)
  - [第五步，注册到 ToolRegistry。](#)
- [05、系统提示词升级](#)
- [06、场景实测](#)
- [07、PaiCLI 如何写到简历上](#)



Agent 可以主动去打开浏览器，能力就有了大幅提升。



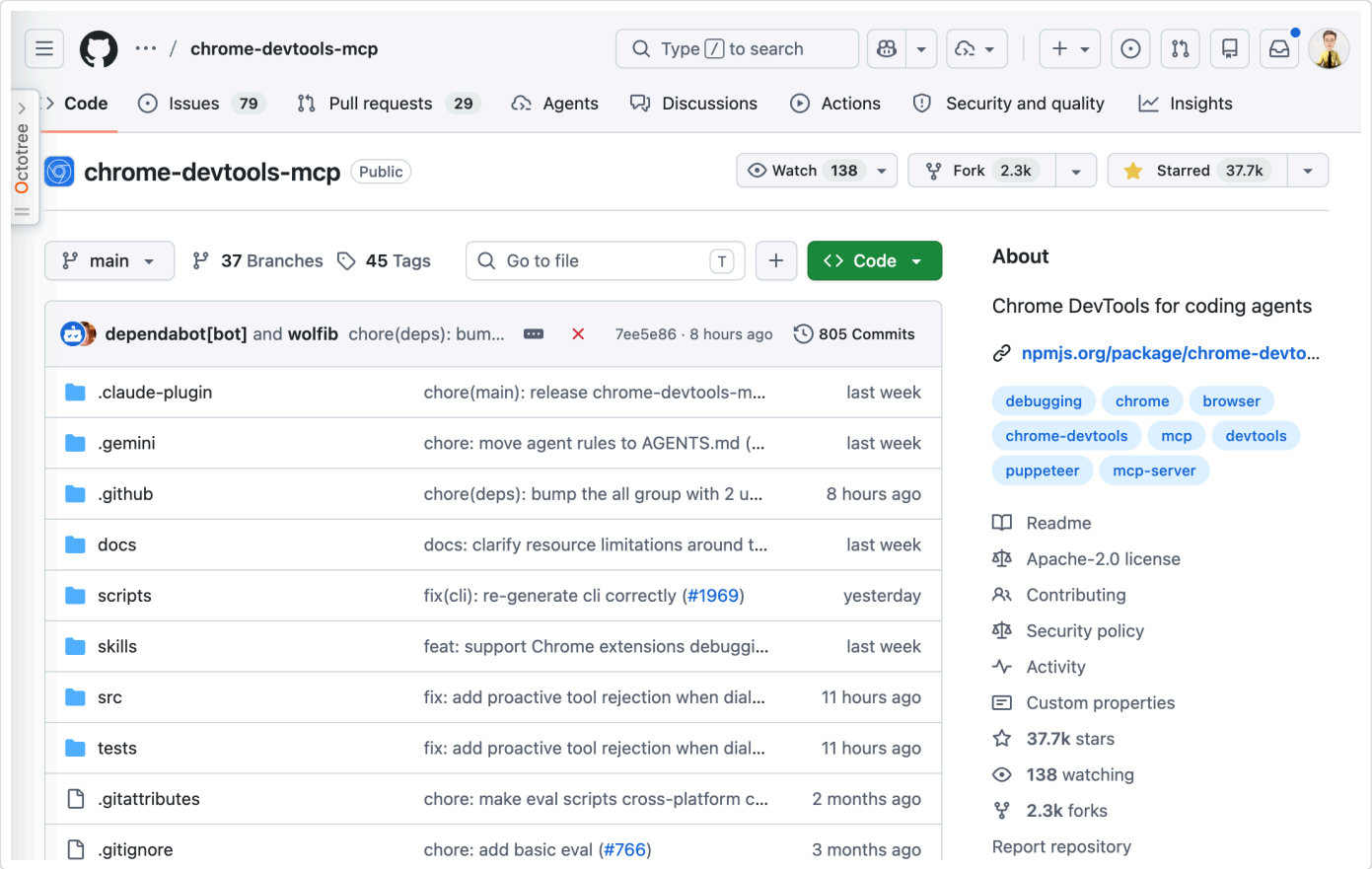
## 01、Chrome DevTools MCP 是什么

Chrome DevTools 是 Google 官方提供的一个 MCP 工具。

### 目录

- [01、Chrome DevTools MCP ...](#)
- [02、CDP 原理与消息流转](#)
- [03、默认接入与启动体验](#)
- [04、从 mcp.json 到工具注册](#)
  - [第一步，读取配置。](#)
  - [第二步，启动 server 进程。](#)
  - [第三步，MCP 握手。](#)
  - [第四步，拉取工具列表。](#)
  - [第五步，注册到 ToolRegistry。](#)
- [05、系统提示词升级](#)
- [06、场景实测](#)
- [07、PaiCLI 如何写到简历上](#)





一共有 28 个工具，按功能分八类。

导航有 6 个：`navigate_page` 开链接、`new_page` 建新标签页、`select_page` 切换标签、`close_page` 关页面、`list_pages` 看开了哪些标签、`wait_for` 等指定元素或文本出现。

## Tools

If you run into any issues, checkout our [troubleshooting guide](#).

- **Input automation (9 tools)**
  - [click](#)
  - [drag](#)
  - [fill](#)
  - [fill\\_form](#)
  - [handle\\_dialog](#)
  - [hover](#)
  - [press\\_key](#)
  - [type\\_text](#)
  - [upload\\_file](#)
- **Navigation automation (6 tools)**
  - [close\\_page](#)
  - [list\\_pages](#)
  - [navigate\\_page](#)
  - [new\\_page](#)
  - [select\\_page](#)
  - [wait\\_for](#)
- **Emulation (2 tools)**
  - [emulate](#)
  - [resize\\_page](#)
- **Performance (3 tools)**
  - [performance\\_analyze\\_insight](#)
  - [performance\\_start\\_trace](#)

版权默认是二条狗

## 目录

- [01、Chrome DevTools MCP ...](#)
- [02、CDP 原理与消息流转](#)
- [03、默认接入与启动体验](#)
- [04、从 mcp.json 到工具注册](#)
  - [第一步，读取配置。](#)
  - [第二步，启动 server 进程。](#)
  - [第三步，MCP 握手。](#)
  - [第四步，拉取工具列表。](#)
  - [第五步，注册到 ToolRegistry。](#)
- [05、系统提示词升级](#)
- [06、场景实测](#)
- [07、PaiCLI 如何写到简历上](#)

模拟用户动作有：`click` 点元素、`fill` 填单个输入框、`fill_form` 一次性填整个表单、`type_text` 模拟键盘输入、`press_key` 按快捷键、`hover` 悬停、`drag` 拖拽、`handle_dialog` 处理弹窗、`upload_file` 传文件。

还有 `take_snapshot` 拿 DOM 文本、`take_screenshot` 截图、`evaluate_script` 执行 JavaScript、`list_console_messages` 看浏览器控制台报错等等。

为什么选官方的 Chrome DevTools MCP，而不是 Playwright 或者 Puppeteer？

三个原因。

第一，Google 官方出品，不需要我们再维护一套浏览器驱动。

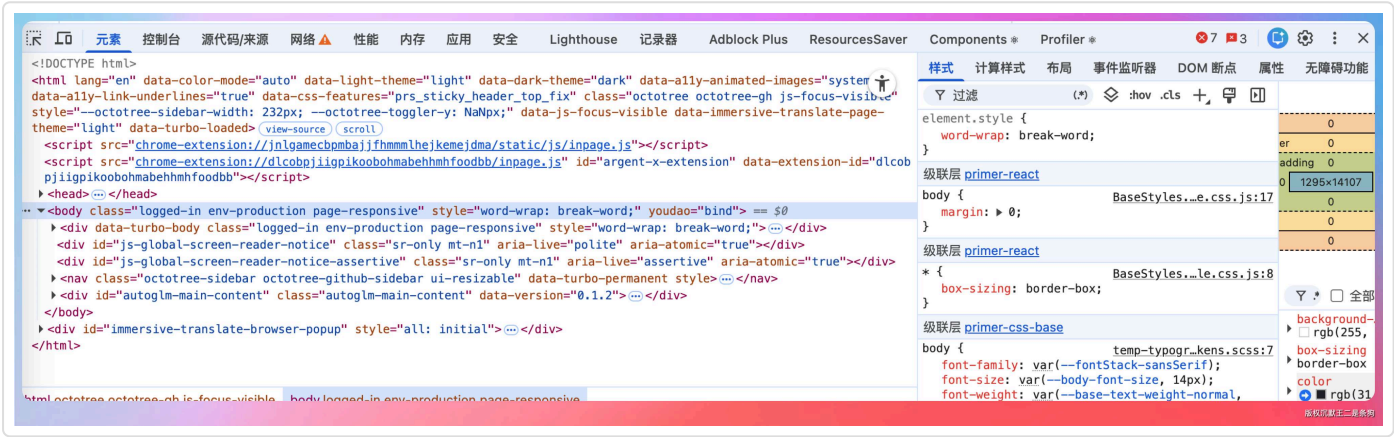
第二，支持 `--isolated=true` 模式，每次启动用临时 user-data-dir，不占用用户的日常 Chrome profile。

第三，原生支持 `--browser-url` 参数，可以复用已打开的 Chrome，为后面的 CDP 会话复用打下基础。

## 02、CDP 原理与消息流转

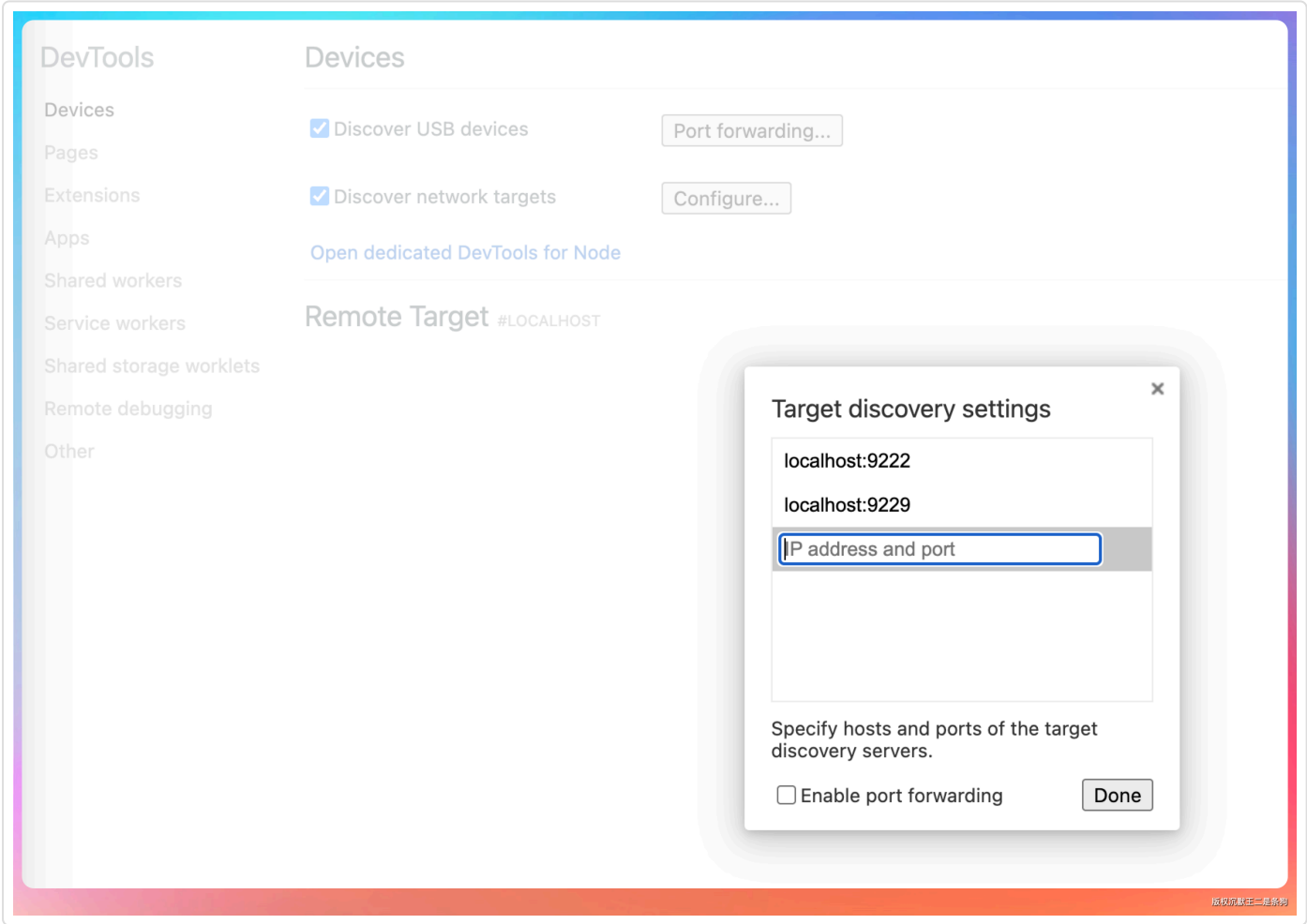
Chrome DevTools MCP 能操控浏览器，靠的是 CDP 协议。

CDP 全称 Chrome DevTools Protocol，是 Chromium 团队提供的一套远程调试协议。



我们平时在 Chrome 里按 F12 打开开发者工具，DevTools 面板和浏览器之间通信用的就是这套协议。只不过 DevTools 面板是给人用的，而 CDP 是给应用程序用的，比如说 Agent。

CDP 的通信方式是 WebSocket。

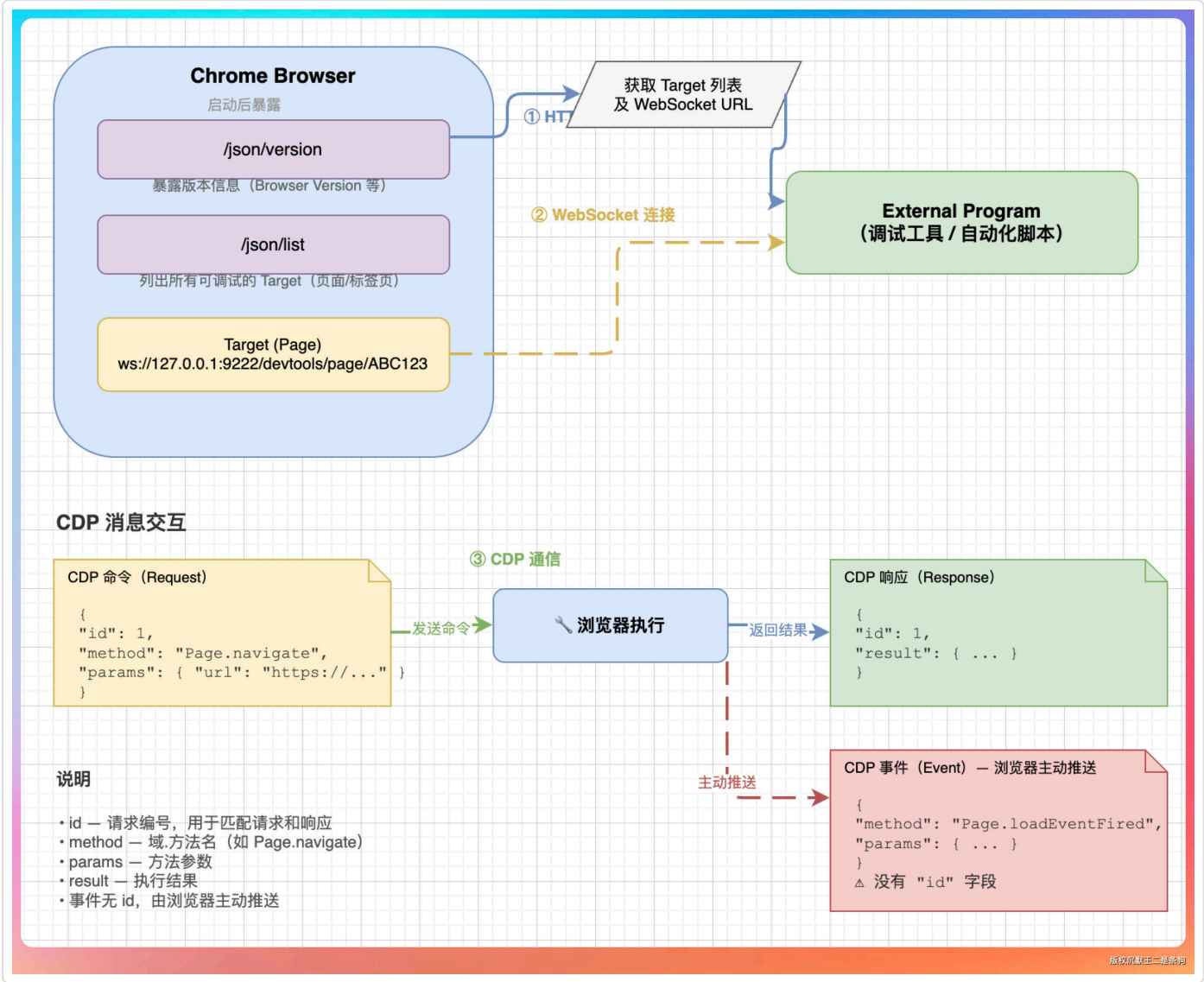


### 目录

- 01、Chrome DevTools MCP ...
- 02、CDP 原理与消息流转
- 03、默认接入与启动体验
- 04、从 mcp.json 到工具注册
  - 第一步，读取配置。
  - 第二步，启动 server 进程。
  - 第三步，MCP 握手。
  - 第四步，拉取工具列表。
  - 第五步，注册到 ToolRegistry。
- 05、系统提示词升级
- 06、场景实测
- 07、PaiCLI 如何写到简历上

Chrome 启动的时候加一个 `--remote-debugging-port=9222` 参数，就会在 9222 端口开一个 WebSocket 服务。外部程序连上这个 WebSocket，就能发命令、接收事件，实现双向实时通信。

具体建立连接的过程是这样的：



目录

- 01、Chrome DevTools MCP ...
- 02、CDP 原理与消息流转
- 03、默认接入与启动体验
- 04、从 mcp.json 到工具注册
  - 第一步，读取配置。
  - 第二步，启动 server 进程。
  - 第三步，MCP 握手。
  - 第四步，拉取工具列表。
  - 第五步，注册到 ToolRegistry。
- 05、系统提示词升级
- 06、场景实测
- 07、PaiCLI 如何写到简历上

Chrome 启动后，先在 `/json/version` 端点暴露版本信息，再在 `/json/list` 端点列出所有可调试的页面（CDP 里叫 target）。

每个 target 都有一个 WebSocket URL，格式类似 `ws://127.0.0.1:9222/devtools/page/ABC123`。外部程序拿到这个 URL 后建立 WebSocket 连接，然后就可以发 CDP 命令了。

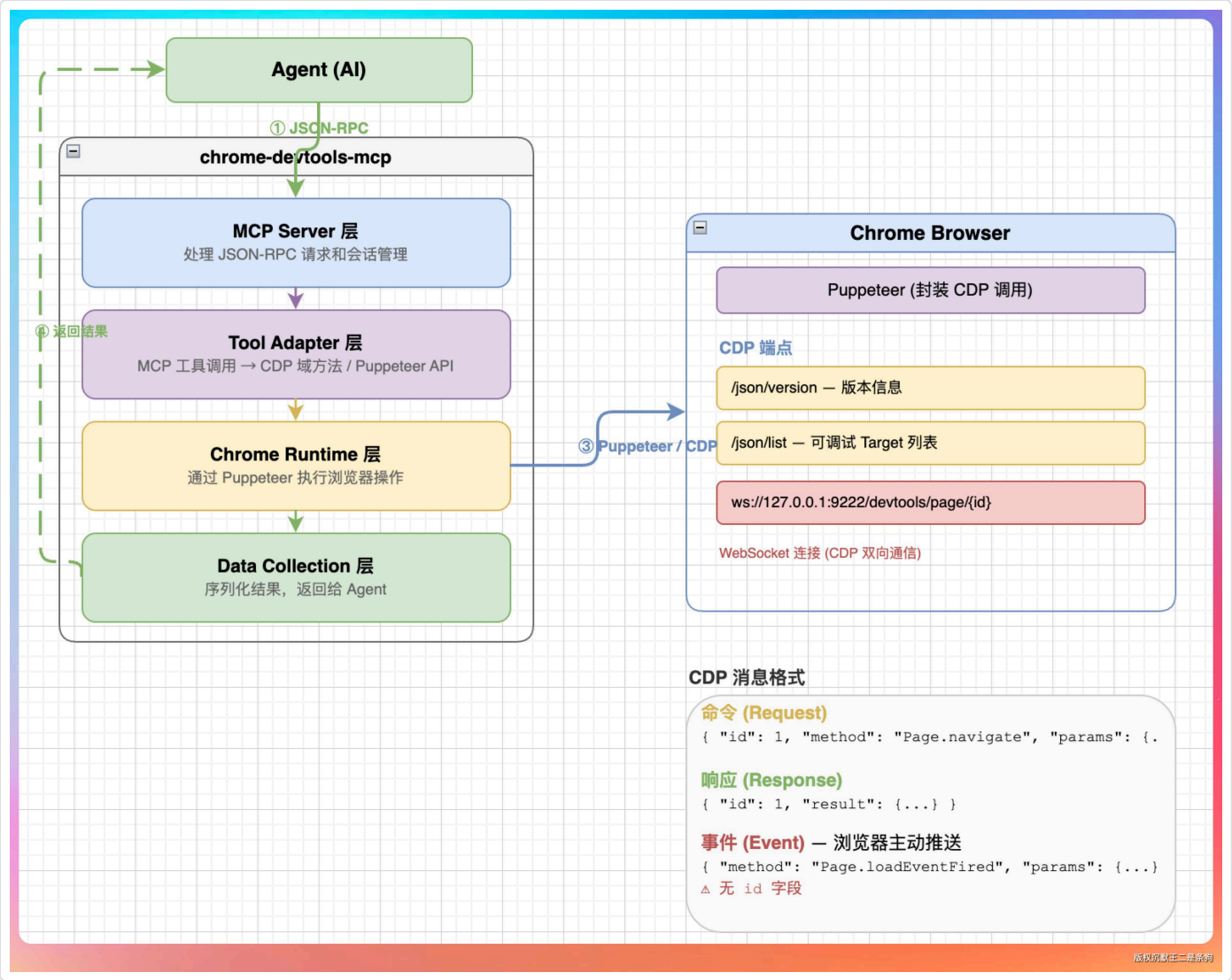
每条 CDP 命令都是一个 JSON 消息，包含 `id`（请求编号）、`method`（域.方法名）、`params`（参数）。浏览器执行完之后返回一个同样带 `id` 的 JSON 响应，包含 `result`（结果）。如果是事件通知，浏览器主动推过来的消息里没有 `id`，而是带 `method` 和 `params`，比如 `Page.loadEventFired`。

chrome-devtools-mcp 的 Puppeteer 把这套底层的 WebSocket 收发都封装好了。

Agent 调 MCP 工具的时候，不需要自己拼 JSON 消息，也不需要管理 WebSocket 连接。Puppeteer 内部还处理了消息的序列号、超时重发、连接断开后的自动重连，这些脏活累活 Agent 一概不用操心。







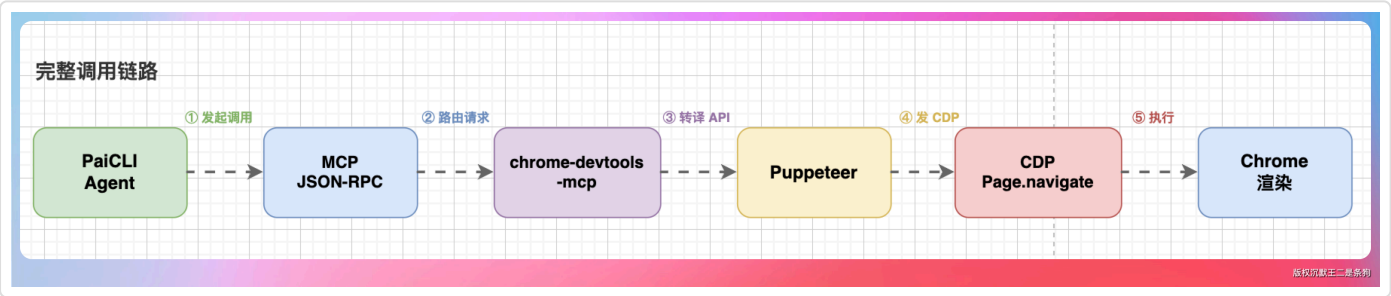
目录

- 01、Chrome DevTools MCP ...
- 02、CDP 原理与消息流转
- 03、默认接入与启动体验
- 04、从 mcp.json 到工具注册
  - 第一步，读取配置。
  - 第二步，启动 server 进程。
  - 第三步，MCP 握手。
  - 第四步，拉取工具列表。
  - 第五步，注册到 ToolRegistry。
- 05、系统提示词升级
- 06、场景实测
- 07、PaiCLI 如何写到简历上

常用的工具和 CDP 域的对应关系大致是这样的：`navigate_page` 和 `new_page` 走 Page 域，`click`、`fill`、`type_text`、`press_key` 走 Input 域，`evaluate_script` 走 Runtime 域，`take_screenshot` 走 Page 域的 `captureScreenshot` 方法。

而 `take_snapshot` 比较特殊，它走的是 Accessibility 域，拿到的是页面的可访问性树 (Accessibility Tree)，这是浏览器为了辅助功能 (比如屏幕阅读器) 维护的一份结构化文本表示，比原始 HTML 更干净，比截图更可读。

Agent 调 `navigate_page` 时，请求路径是这样的：



再比如 `take_snapshot`，它走的是 CDP 的 Accessibility 域或者 DOM 域，把页面的可访问性树或者 DOM 结构提取成纯文本，直接返回给 LLM 阅读。

而 `take_screenshot` 走的是 CDP 的 `Page.captureScreenshot`，返回的是 PNG 图片的二进制数据。

整个消息流是双向的。

Agent 发命令给浏览器，浏览器也会主动推事件回来。比如页面加载完成会推 `Page.loadEventFired`，网络请求完成会推 `Network.requestWillBeSent`。

chrome-devtools-mcp 内部会订阅这些事件，配合 `wait_for` 工具让 Agent 能等特定条件出现再继续操作。

理解了 CDP 这一层，后面再看 chrome-devtools-mcp 的工具设计就很好懂了：每个工具其实就是对一组 CDP 方法的高级封装，加上了参数校验、错误处理和结果格式化，让 Agent 不需要直接跟 CDP 的底层细节打交道。

03、默认接入与启动体验

chrome-devtools 在 PaiCLI 里是默认 enabled 的。

~/.paicli/mcp.json 如果不存在，启动时会自动创建一份默认模板，里面就包含 chrome-devtools 的配置：

json 复制代码

```
{
  "mcpServers": {
    "chrome-devtools": {
      "command": "npx",
      "args": ["-y", "chrome-devtools-mcp@latest", "--isolated=true"]
    }
  }
}
```



--isolated=true 告诉 server 每次启动都用一个全新的临时用户数据目录，不要去碰用户日常的 Chrome 书签、历史、Cookie。

这个隔离机制值得展开说一说。

--isolated=true 的底层原理是每次启动 Chrome 的时候，Puppeteer 会创建一个操作系统临时目录下的子目录作为 --user-data-dir，比如 /tmp/puppeteer-dev\_profile-abc123。

这个目录里存的是浏览器的所有本地数据：Cookie、localStorage、sessionStorage、IndexedDB、缓存、登录态。浏览器关闭之后，这个临时目录会被自动清理掉，不留痕迹。

隔离的好处是：Agent 在浏览器里做的所有操作，不会污染用户日常的 Chrome。即使 Agent 访问了恶意网站，Cookie 被篡改了，关掉浏览器就什么都清了。

但隔离也有代价：登录态没法共用。

目录

- 01、Chrome DevTools MCP ...
- 02、CDP 原理与消息流转
- 03、默认接入与启动体验
- 04、从 mcp.json 到工具注册
  - 第一步，读取配置。
  - 第二步，启动 server 进程。
  - 第三步，MCP 握手。
  - 第四步，拉取工具列表。
  - 第五步，注册到 ToolRegistry。
- 05、系统提示词升级
- 06、场景实测
- 07、PaiCLI 如何写到简历上





如果你让 Agent 登录了某个网站，下次再开浏览器，登录态没了，又得重新登录。

这正是之后我们要解决的问题：用 `--browser-url` 参数连接到用户日常打开的 Chrome，复用已有的登录状态。`--browser-url` 的用法是 `--browser-url=http://127.0.0.1:9222`，直接连上一个已经打开的 Chrome 实例，不走 `isolated` 模式。

另外，macOS 用户首次运行可能会碰到系统权限弹窗。

Chrome 在 `isolated` 模式下启动时，macOS 可能会请求“辅助功能”或“屏幕录制”权限，这是操作系统级别的安全策略，必须手动点允许，否则 Chrome 启动会失败。如果碰到启动超时，先检查一下系统偏好设置里有没有待审批的权限请求。

可以用 `/mcp disable chrome-devtools` 随时关闭。

当然了，默认 `enabled` 会让 PaiCLI 首次启动特别慢。

为了减轻用户的等待焦虑，`McpServerManager.startAll()` 里另起了一个 `daemon` 进度来打印线程，每 5 秒检查一次还没 `ready` 的 `server`，把等待时长实时打印出来：

undefined 复制代码

```
🚀 启动 MCP server (5 个) ...
✓ filesystem      stdio    14 工具    1.2s
⌚ chrome-devtools stdio    启动中... (首次需拉包 + 启动 Chrome)
✓ zread           http     3 工具    0.8s
⌚ chrome-devtools stdio    启动中... (已等待 12s)
✓ chrome-devtools stdio    28 工具   18.5s
5/5 就绪, 共 60 个 MCP 工具
```

## 目录

- [01、Chrome DevTools MCP ...](#)
- [02、CDP 原理与消息流转](#)
- [03、默认接入与启动体验](#)
- [04、从 mcp.json 到工具注册](#)
  - [第一步，读取配置。](#)
  - [第二步，启动 server 进程。](#)
  - [第三步，MCP 握手。](#)
  - [第四步，拉取工具列表。](#)
  - [第五步，注册到 ToolRegistry。](#)
- [05、系统提示词升级](#)
- [06、场景实测](#)
- [07、PaiCLI 如何写到简历上](#)



目录

- [01、Chrome DevTools MCP ...](#)
- [02、CDP 原理与消息流转](#)
- [03、默认接入与启动体验](#)
- [04、从 mcp.json 到工具注册](#)
  - [第一步，读取配置。](#)
  - [第二步，启动 server 进程。](#)
  - [第三步，MCP 握手。](#)
  - [第四步，拉取工具列表。](#)
  - [第五步，注册到 ToolRegistry。](#)
- [05、系统提示词升级](#)
- [06、场景实测](#)
- [07、PaiCLI 如何写到简历上](#)

04、从 mcp.json 到工具注册

前面讲了 CDP 原理和启动体验，这一节我们把整个接入路径从配置到工具注册串起来，看看 PaiCLI 内部到底做了什么。

第一步，读取配置。

Main.java 启动时会检测 ~/.paicli/mcp.json 是否存在。

如果文件不存在，就用默认模板创建一份，里面包含 chrome-devtools 的配置。如果文件已经存在但缺 chrome-devtools 条目，只打印一行提示，不会擅自改用户的配置文件。

第二步，启动 server 进程。

McpServerManager.startAll() 遍历 mcpServers 里所有非 disabled 的条目，对每个 server 启动一个子进程。

chrome-devtools 的 command 是 npx，args 是 ["-y", "chrome-devtools-mcp@latest", "--isolated=true"]。

npx 会先在本地缓存里找，找不到就从 npm 拉取，然后执行。这就是首次启动特别慢的原因。

第三步，MCP 握手。

子进程起来之后，McpClient.initialize() 发送 JSON-RPC initialize 请求，等 server 回复 capabilities。

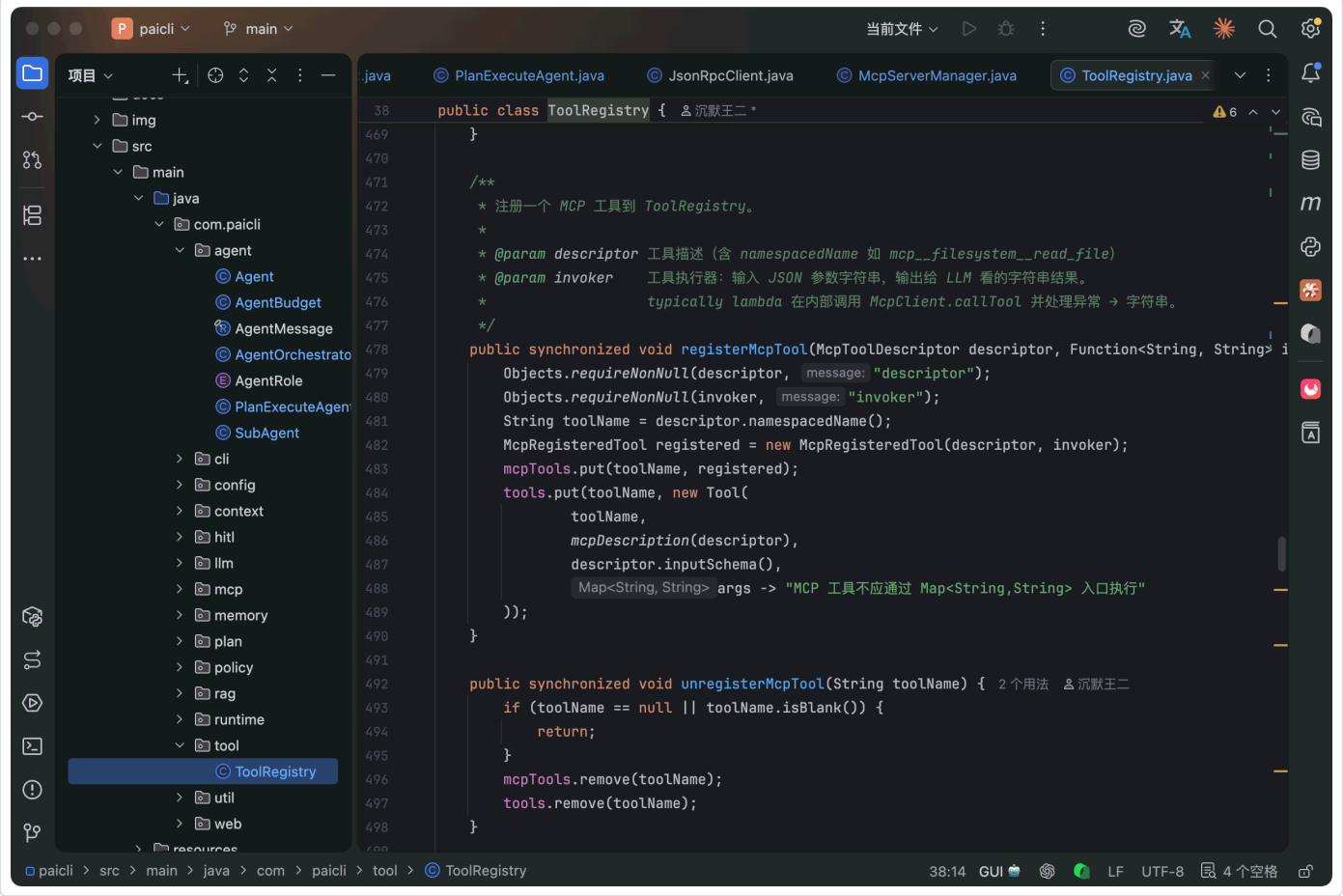
第四步，拉取工具列表。

chrome-devtools server 在 capabilities 里声明了 tools 能力，PaiCLI 就调 tools/list 拿到 28 个工具的 schema 定义，每个工具包含名称、描述和参数格式。

第五步，注册到 ToolRegistry。

28 个工具按 mcp\_{server}\_{tool} 的格式注册，比如 mcp\_chrome-devtools\_navigate\_page、mcp\_chrome-devtools\_take\_snapshot。





这些工具跟内置工具和之前注册的其他 MCP 工具在同一个 Registry 里统一管理，HITL 审批、AuditLog 记录一个都不少。

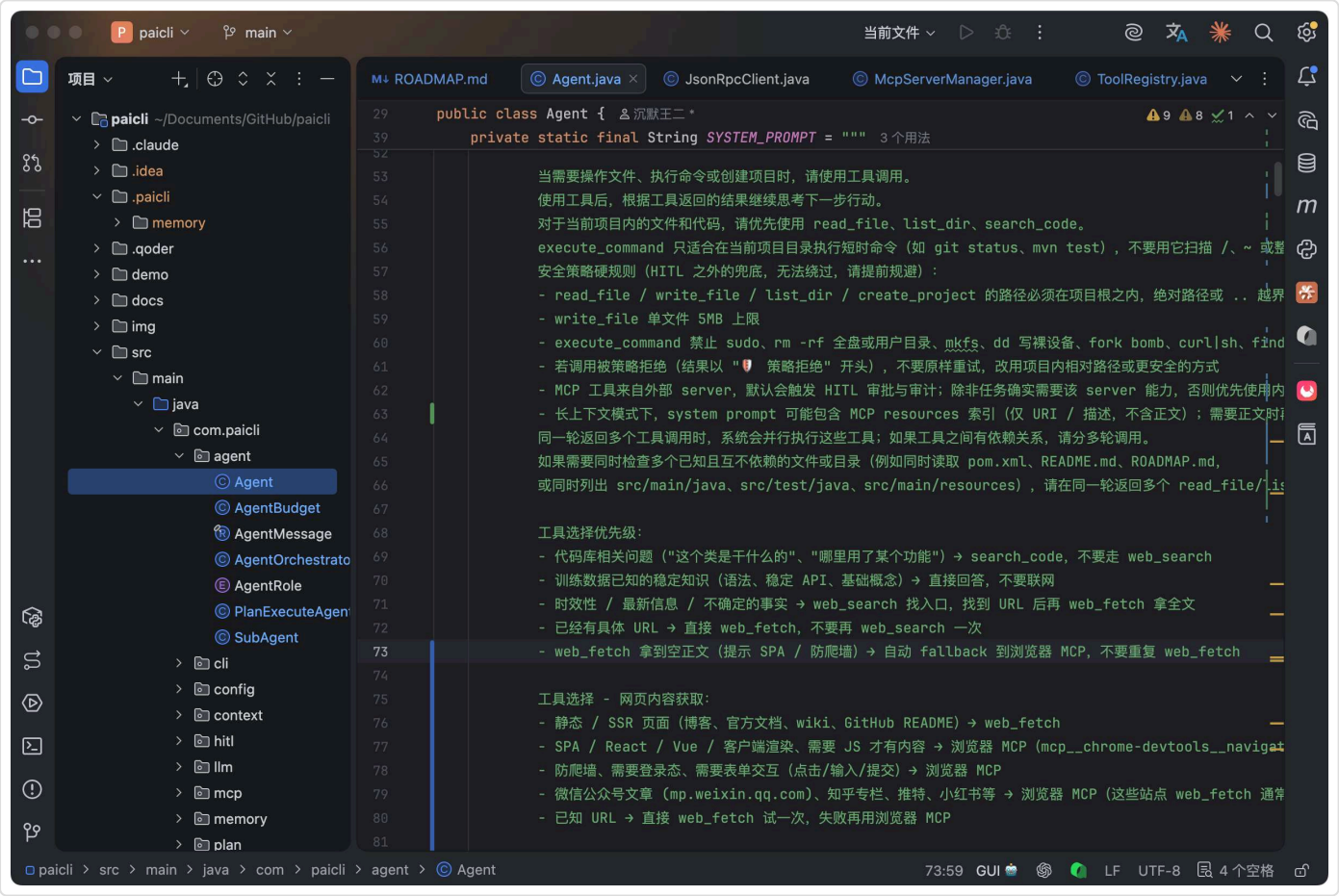
有一点值得说：chrome-devtools-mcp 的浏览器不是连接时就启动的。

它是在 Agent 第一次调用浏览器相关工具时，才会自动拉起 Chrome 实例。这意味着如果用户只是启动了 PaiCLI 但没让 Agent 操作浏览器，Chrome 进程是不会出现的，不占用资源。

## 05、系统提示词升级

光把工具注册进去还不够，得让 Agent 知道什么时候用浏览器、什么时候用 web\_fetch。

否则 Agent 遇到微信内容还是先去调 web\_fetch，失败之后再重试，很浪费 token 和时间。



于是我们在系统提示词里加了「web\_fetch vs 浏览器 MCP」决策表：Agent.SYSTEM\_PROMPT、PlanExecuteAgent.EXECUTION\_PROMPT、SubAgent.WORKER\_PROMPT。

这套提示词相当于给 Agent 写了一份“浏览器操作手册”。

## 06、场景实测

代码写完了，Prompt 也调好了，该上手验证了。

### 目录

[01、Chrome DevTools MCP ...](#)

[02、CDP 原理与消息流转](#)

[03、默认接入与启动体验](#)

[04、从 mcp.json 到工具注册](#)

[第一步，读取配置。](#)

[第二步，启动 server 进程。](#)

[第三步，MCP 握手。](#)

[第四步，拉取工具列表。](#)

[第五步，注册到 ToolRegistry。](#)

[05、系统提示词升级](#)

[06、场景实测](#)

[07、PaiCLI 如何写到简历上](#)

提示词：测试链接是 [https://mp.weixin.qq.com/s/RB7kF\\_BbsJZ5\\_Hmu9PxWdg](https://mp.weixin.qq.com/s/RB7kF_BbsJZ5_Hmu9PxWdg)，帮我看下这篇文章讲了什么。

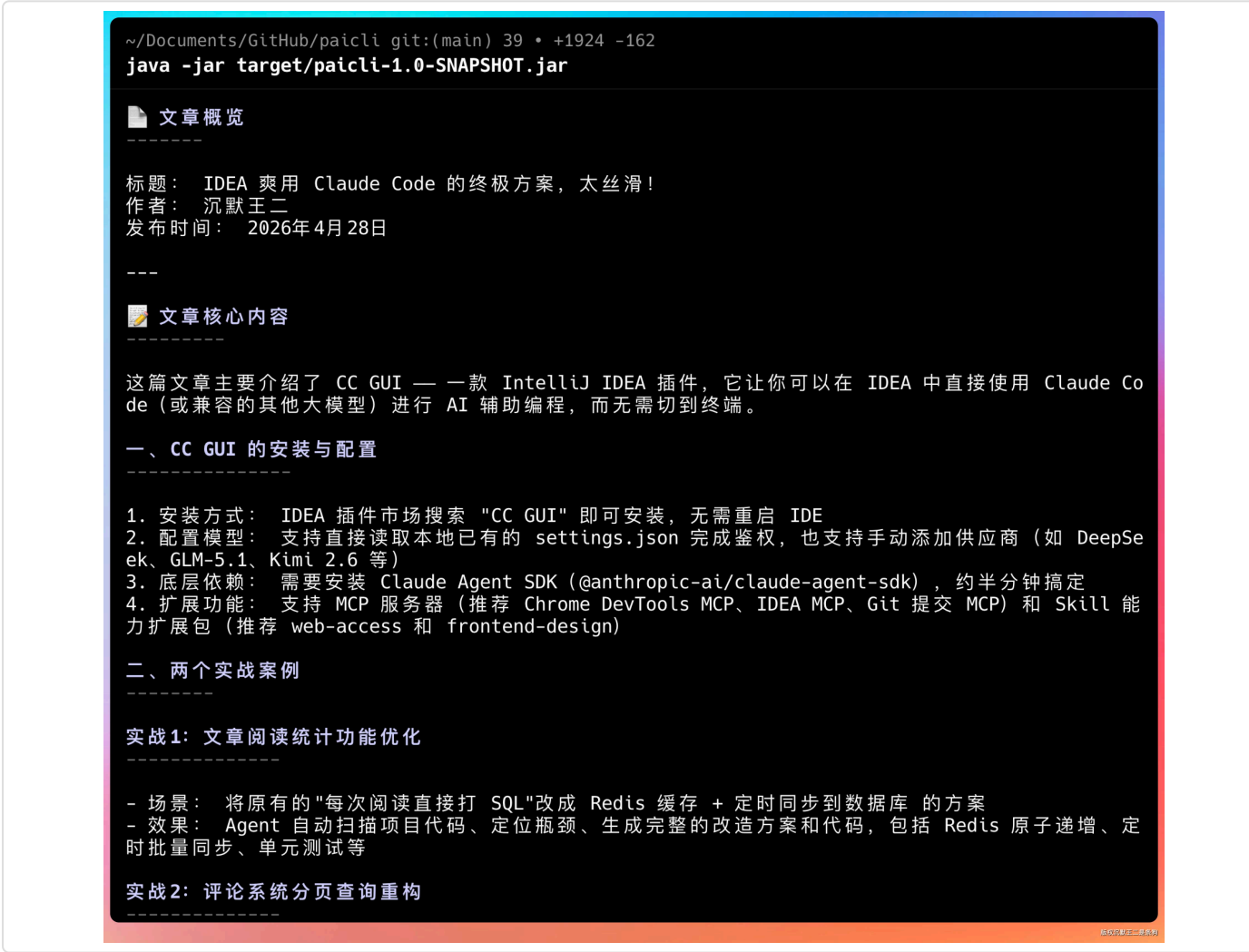


目录

- [01、Chrome DevTools MCP ...](#)
- [02、CDP 原理与消息流转](#)
- [03、默认接入与启动体验](#)
- [04、从 mcp.json 到工具注册](#)
  - [第一步，读取配置。](#)
  - [第二步，启动 server 进程。](#)
  - [第三步，MCP 握手。](#)
  - [第四步，拉取工具列表。](#)
  - [第五步，注册到 ToolRegistry。](#)
- [05、系统提示词升级](#)
- [06、场景实测](#)
- [07、PaiCLI 如何写到简历上](#)

从 llm 的思考过程就能看得出来，用户想要阅读微信生态的内容，我会先尝试用 web\_fetch，如果不行，我会使用 Chrome 的 MCP。

实际上，提示词里已经讲了，阅读微信生态的内容需要用浏览器的 MCP，所以直接调用了 `mcp__chrome-devtools__new_page`，然后等内容加载后，通过 `take_snapshot` 去拿正文的 DOM，最后总结输出。



完全没问题。

并且确实新开了一个 Chrome 去阅读这篇内容。

about:blank

IDEA 爽用 Claude Code 的终极方案，太丝滑！

mp.weixin.qq.com/s/RB7kF\_BbsJZ5\_Hmu9PxWdg

Chrome 正受到自动测试软件的控制

# IDEA 爽用 Claude Code 的终极方案，太丝滑！

原创

沉默王二

沉默王二

2026年4月28日 14:04

河南

大家好，我是二哥呀。

之前有读者在评论区安利了一款叫 CC GUI 的 IntelliJ IDEA 插件。

可以在idea中爽用Claude Code。

刚好 DeepSeek V4 pro 大降价，我就想着，能不能在 CC GUI 中配置上 DeepSeek V4 的 API key，然后爽用起来。

| 模型                      | deepseek-v4-flash <sup>(1)</sup>   | deepseek-v4-pro |
|-------------------------|------------------------------------|-----------------|
| BASE URL (OpenAI 格式)    | https://api.deepseek.com           |                 |
| BASE URL (Anthropic 格式) | https://api.deepseek.com/anthropic |                 |
| 模型版本                    | DeepSeek-V4-Flash                  | DeepSeek-V4-Pro |

沉默王二

👍 赞

🔗 分享

❤️ 推荐

💬 写留言

第二个提示词是：截图看一下 paicoding.com 的首页

你：截图看一下paicoding.com的首页

运行中按 ESC 取消当前任务。

调用 MCP 工具 chrome-devtools.navigate\_page

L {"type":"url","url":"https://paicoding.com","timeout":15000}

思考过程

The user wants me to take a screenshot of the paicoding.com homepage. I'll use the browser t o navigate to the site and take a screenshot.

Page loaded. Now let me take a screenshot.

回复

调用 MCP 工具 chrome-devtools.take\_screenshot

L {"fullPage":false,"format":"png"}

页面已加载，正在截图：

The screenshot was taken but it seems the image content isn't being shown to me in text form . Let me take a screenshot and save it to a file so the user can see it. Let me also take a snapshot to understand the page content.

回复

调用 MCP 工具 chrome-devtools.take\_screenshot

L {"filePath":"paicoding-homepage.png","fullPage":true,"format":"png"}

调用 MCP 工具 chrome-devtools.take\_snapshot

L {}

截图已生成，但内容可能只显示在我的视觉范围内。让我同时保存截图文件并获取页面文本快照，这样你可以更清楚地了解页面内容。

这时候会调用 MCP 工具 chrome-devtools.take\_screenshot，然后再传给 llm 去读内容。

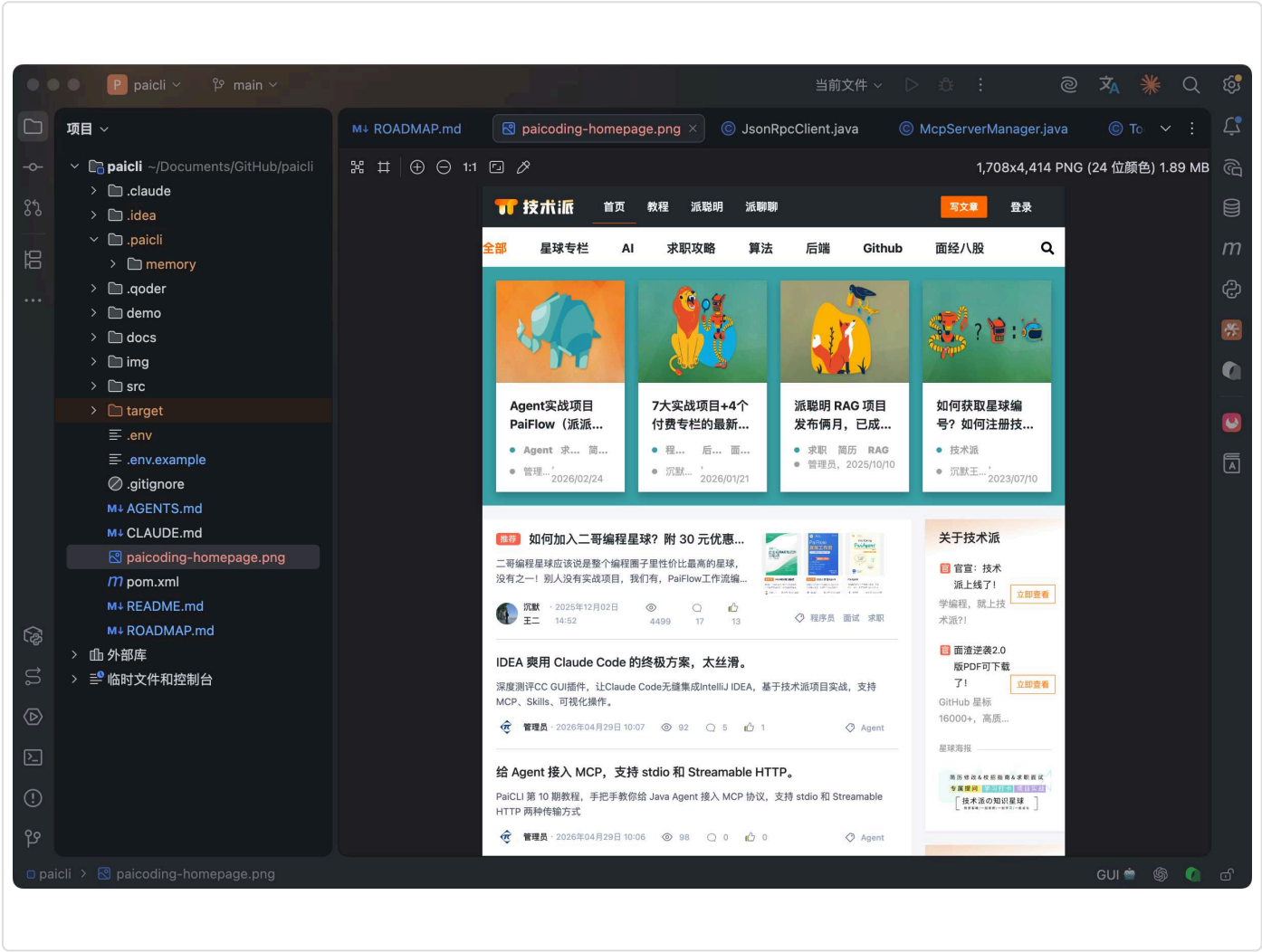
## 目录

- 01、Chrome DevTools MCP ...
- 02、CDP 原理与消息流转
- 03、默认接入与启动体验
- 04、从 mcp.json 到工具注册
  - 第一步，读取配置。
  - 第二步，启动 server 进程。
  - 第三步，MCP 握手。
  - 第四步，拉取工具列表。
  - 第五步，注册到 ToolRegistry。
- 05、系统提示词升级
- 06、场景实测
- 07、PaiCLI 如何写到简历上





项目的根目录确实也能看到截图。



## 07、PaiCLI 如何写到简历上

**项目名称：**PaiCLI — Browser-Capable Agent CLI

**项目简介：**基于 Java 实现的 AI Agent 命令行工具，接入 Chrome DevTools MCP server，使 Agent 具备完整浏览器自动化能力，覆盖导航、输入、DOM 快照、截图、网络请求监控等 28 个工具。

**技术栈：**Java 21、MCP Protocol、Chrome DevTools Protocol、JSON-RPC 2.0、ConcurrentHashMap、CompletableFuture

**核心职责：**

- 接入 chrome-devtools-mcp，设计 `--isolated=true` 安全隔离策略，实现 mcp.json 自动模板创建
- 设计 web\_fetch → 浏览器 MCP 自动 fallback 机制，当 web\_fetch 因 SPA / 反爬墙 / 客户端渲染返回空正文时，Agent 自动切换到 Chrome DevTools MCP 通过 take\_snapshot 拿取 DOM 文本，覆盖

## 目录

- [01、Chrome DevTools MCP ...](#)
- [02、CDP 原理与消息流转](#)
- [03、默认接入与启动体验](#)
- [04、从 mcp.json 到工具注册](#)
  - [第一步，读取配置。](#)
  - [第二步，启动 server 进程。](#)
  - [第三步，MCP 握手。](#)
  - [第四步，拉取工具列表。](#)
  - [第五步，注册到 ToolRegistry。](#)
- [05、系统提示词升级](#)
- [06、场景实测](#)
- [07、PaiCLI 如何写到简历上](#)



微信公众号、知乎专栏、掘金等 web\_fetch 不可达场景

- 设计 MCP server 启动进度打印线程，每 5 秒刷新未就绪 server 等待时长



真诚点赞 诚不我欺



讨论应以学习和精进为目的。请勿发布不友善或者负能量的内容，与人为善，比聪明更重要！



0/512

评论

目录

[01、Chrome DevTools MCP ...](#)

[02、CDP 原理与消息流转](#)

[03、默认接入与启动体验](#)

[04、从 mcp.json 到工具注册](#)

[第一步，读取配置。](#)

[第二步，启动 server 进程。](#)

[第三步，MCP 握手。](#)

[第四步，拉取工具列表。](#)

[第五步，注册到 ToolRegistry。](#)

[05、系统提示词升级](#)

[06、场景实测](#)

[07、PaiCLI 如何写到简历上](#)